WEEK 7 ASSESSMENT

WEB APPLICATION SECURITY TESTING WITH BURP SUITE

Cybersecurity Internship Report

Submitted by: Amaan Fayaz
Internship Program: DG Interns Hub – Cybersecurity Internship
Week: 7
Topic: VAPT (Web Application Testing with Burp Suite)
Tools Used: Burp Suite Community Edition, DVWA, and Gobuster

Testing Environment: Local DVWA Lab

Submission Date: 21st october2026

Task 22: Burp Suite Setup & Interception

Step 1: Launching Burp Suite

Burp Suite Community Edition v2026.8 was launched successfully in the Kali Linux environment. A temporary project was created using the default configuration settings. The Proxy module was opened to prepare for intercepting and analyzing HTTP and HTTPS requests between the browser and the target web application.

Initially, Burp Suite displayed a message stating that no proxy listeners were running. Therefore, the proxy listener needed to be enabled before proceeding with web application testing.

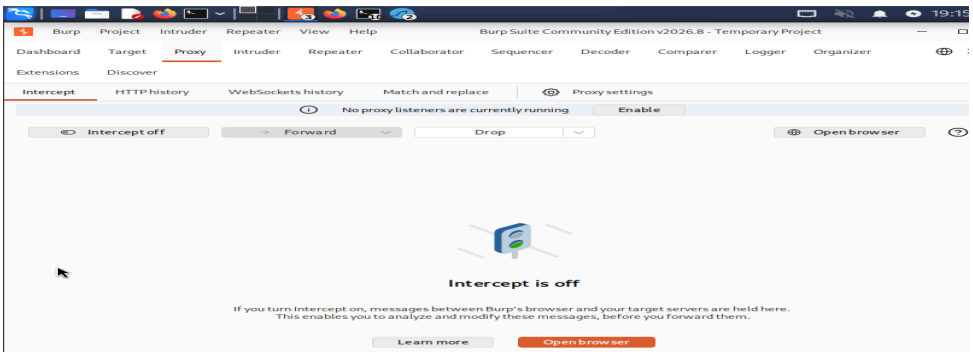


Figure 1: Launching Burp Suite Community Edition in Kali Linux

Step 2: Proxy Listener Configuration

The proxy listener configuration was checked in the Burp Suite Community Edition. The listener was configured to use the local loopback address 127.0.0.1 and port 8080. The proxy listener status was examined before proceeding to browser-based testing.

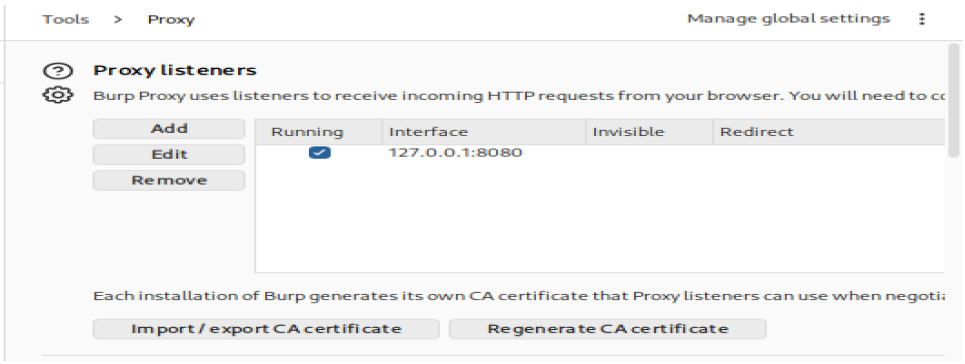


Figure 2: Proxy Listener Configuration in Burp Suite Community Edition.

Step 3: Intercept HTTP Requests

Step 3.1: Checking the DVWA Installation

The DVWA installation was checked using the Kali Linux terminal. The command ls /var/www/html was executed to inspect the default web server directory.

The output displayed two files: index.html and index.nginx-debian.html. No DVWA folder was found in this directory. Therefore, further investigation was required to determine whether DVWA was installed in another location or whether a new installation was necessary.

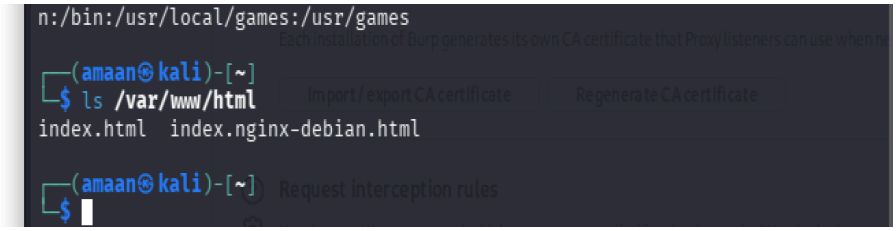


Figure 3: Checking the Web Server Directory for DVWA in Kali Linux

Step 3.2: Check Apache Web Server

The Apache HTTP Server was checked using the Kali Linux terminal. Initially, the service was found to be inactive. The service was subsequently started using the systemctl command. After starting the service, its status was checked again, and the output confirmed that Apache was active and running. This prepared the local environment for hosting and testing a web application.

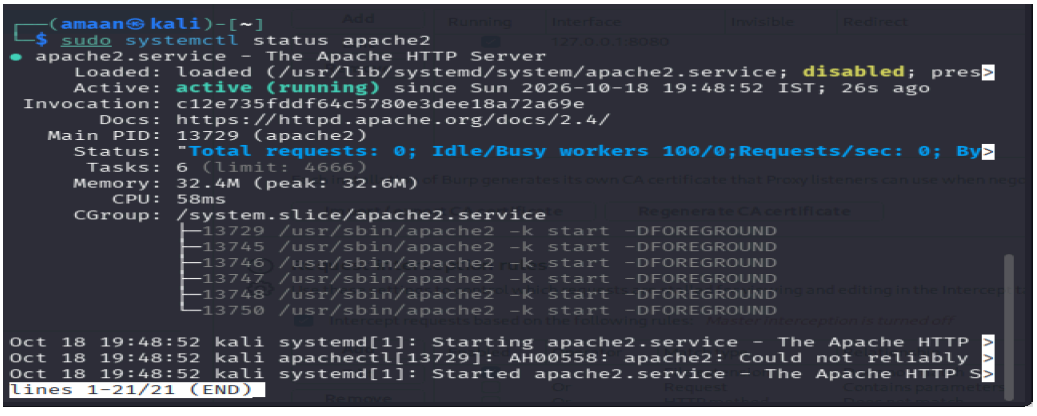


Figure 4: Apache Web Server Running in Kali Linux

Step 3.3: Test the Web Server

The local web server was tested using the curl command in the Kali Linux terminal. The command sent an HTTP request to http://127.0.0.1 and returned the status code HTTP/1.1 200 OK. The response confirmed that the Apache web server was accessible and responding to HTTP requests. This verified that the local web server was available for further configuration and testing.

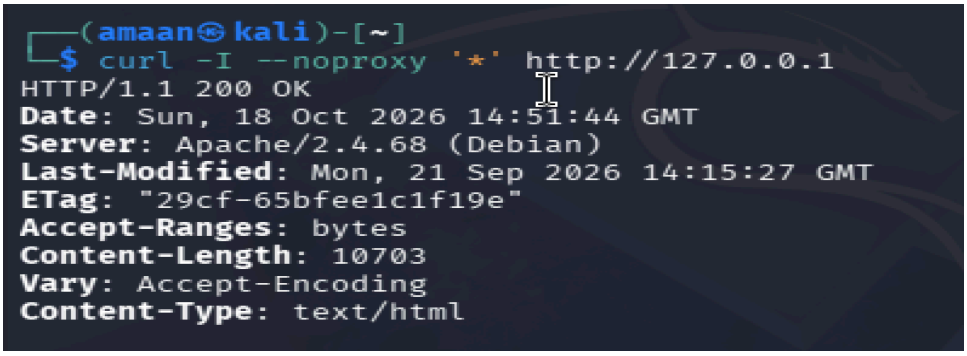


Figure 5: Successful HTTP Response from the Local Apache Web Server

Step 3.4: Open the Web Server Through Burp Browser

An attempt was made to access the local Apache web server through the Burp Suite browser. The browser displayed a connection error for 127.0.0.1:80, although a direct terminal request to the same address had previously returned HTTP 200 OK. Further investigation was therefore required to determine whether the issue was related to proxy configuration or browser connectivity.

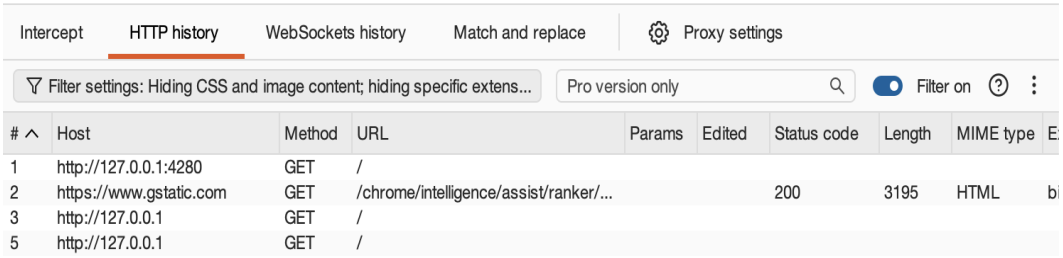


Figure 6: Connection Error Observed in the Burp Suite Browser

Step 3.5: Analyze the Captured HTTP Requests

Burp Suite Community Edition was used to capture and inspect an HTTP request generated by the browser. The request used the GET method to access the root path (/) of the host 127.0.0.1:4280. The request headers contained information about the browser, accepted content types, language preferences, and connection settings.

This practical demonstrated how Burp Suite can capture and display HTTP requests for further analysis during web application security testing. The availability of the target application's response still needed to be verified.

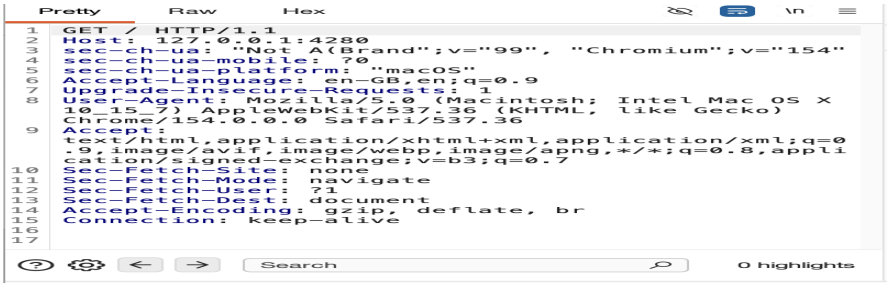


Figure 7: Captured HTTP GET Request in Burp Suite

Step 3.6: Check the Server Response

Burp Suite was used to inspect HTTP requests generated by the browser. A GET request was captured for the local address 127.0.0.1:4280. The request details, including the HTTP method, request path, host and browser headers, were examined in the Pretty view. Initially, the target application could not be reached through the browser. Further investigation revealed that the Apache web server was inactive at one point. The Apache service was subsequently started, and the terminal confirmed that it was listening on port 80. The response from the target application on port 4280 still needs to be verified.



Figure 8: Inspecting the HTTP Request in Burp Suite

Step 3.7: Check the Server Response

The server response was checked using Burp Suite and the Kali Linux terminal. The curl command was used to test whether the local Apache web server was responding to HTTP requests.

The command used was: curl -I --noproxy '*' http://127.0.0.1/

The expected response from a working web server is HTTP/1.1 200 OK. This status code indicates that the server successfully processed the request. Burp Suite's HTTP history was also examined to review captured requests and their corresponding responses. This helped me understand how to monitor HTTP communication and verify server availability during web application security testing.



Figure 8: Inspecting the HTTP Request in Burp Suite

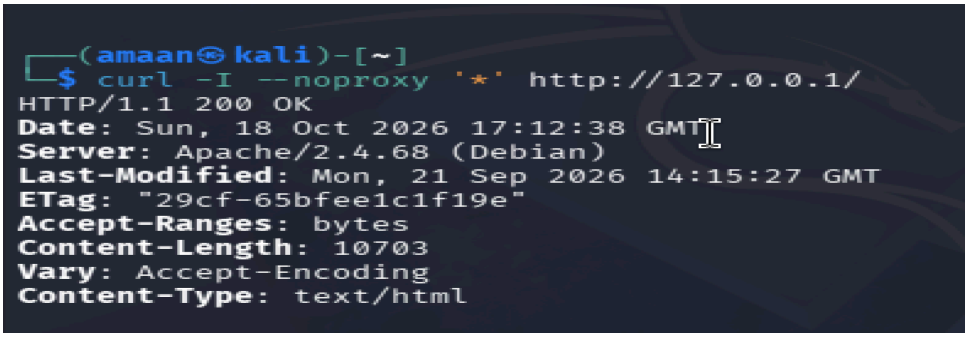


Figure 9: Verifying the Apache Web Server Response Using the curl Command

The server response was checked using the curl command. The HTTP status code 200 OK confirms that the server successfully processed the request.

Step 3.8: Capture and Analyze a Login Request

Objective: The objective of this task is to capture and analyze an HTTP login request using Burp Suite to understand how login credentials and authentication data are transmitted between a browser and a web application.

Procedure: Burp Suite Community Edition was used to inspect HTTP requests generated by the browser. The Proxy feature allows requests to be intercepted and examined before they are forwarded to the web server. The request method, URL, headers, and parameters can be analyzed to understand the login process.

Observation: The HTTP request contains information such as the request method, host, headers, and, when applicable, login form parameters. Examining these details helps identify how authentication requests are handled and highlights the importance of protecting sensitive information.

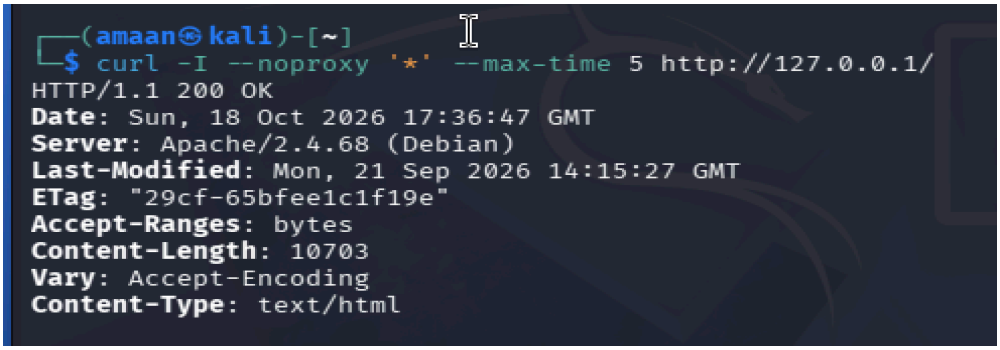


Figure 10: Capturing and Analyzing a Login Request Using Burp Suite

Task 23: Reconnaissance and Web Application Mapping

Web application reconnaissance involves exploring an application to understand its structure, available pages, and communication endpoints. Burp Suite Community Edition was used to observe HTTP requests generated during browsing. The captured requests were reviewed to identify URLs, request methods, and other relevant information about the application's attack surface.

Identifying Application Endpoints

Web application endpoints were identified by navigating through different pages of the DVWA application. Burp Suite was intended to capture and record the HTTP requests generated during this process. Each request can provide information about the requested URL, HTTP method, parameters, and server response.

The main pages explored included the login page, Brute Force, Command Injection, SQL Injection, and Cross-Site Scripting (XSS). These pages represent different functionalities that can be examined during a web application security assessment.

This activity helped me understand how web applications communicate with servers and how identifying endpoints provides a foundation for further vulnerability testing.

Burp Suite was used to inspect HTTP requests associated with the login page. The request details can help identify the HTTP method, URL, headers, and parameters submitted by the form.



Figure 1: DVWA login page showing the authentication form and input fields.

HTTP Request Analysis Using Burp Suite

Burp Suite's Proxy and HTTP history features were used to observe HTTP requests generated while accessing the DVWA application. The captured requests were examined to understand the communication between the browser and the web server.

The HTTP history can display information such as the request method, URL path, response status code, and other request details. This information helps identify the endpoints accessed during reconnaissance.



Figure 2: HTTP request analysis in Burp Suite showing the DVWA login endpoint, request method, host, and HTTP headers.

HTTP Request and Parameter Analysis

The Burp Suite was used to inspect HTTP requests sent to the DVWA web application. The captured request showed the endpoint /dvwa/login.php, the HTTP request method, the host address, and the request headers. This information helps explain how the browser communicates with the web server.

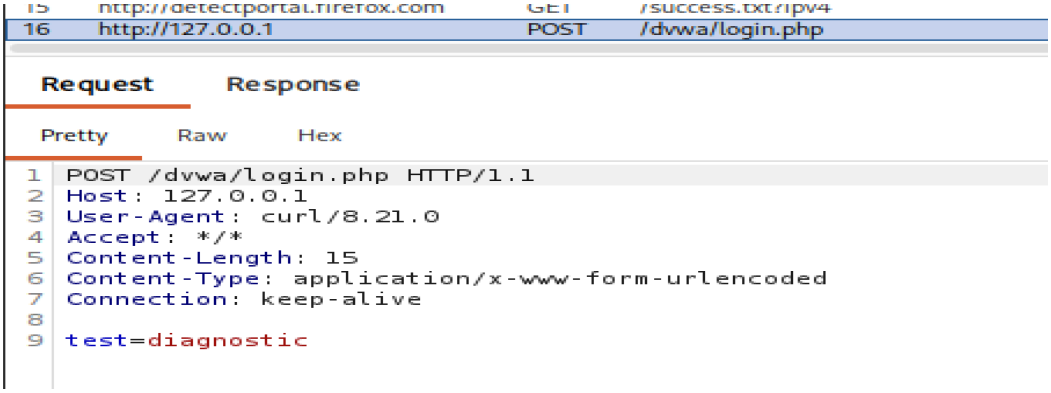


Figure 3: Burp Suite displaying a POST request to the DVWA login endpoint, including HTTP headers and URL-encoded request data.

Identifying Application Endpoints

During the reconnaissance process, the DVWA web application was explored to identify its available pages and functionalities. Burp Suite Community Edition was used to observe HTTP requests generated while navigating through the application. The Target → Site map feature helps organize discovered URLs into a hierarchical structure, making it easier to understand the application's structure and identify endpoints for further security assessment.

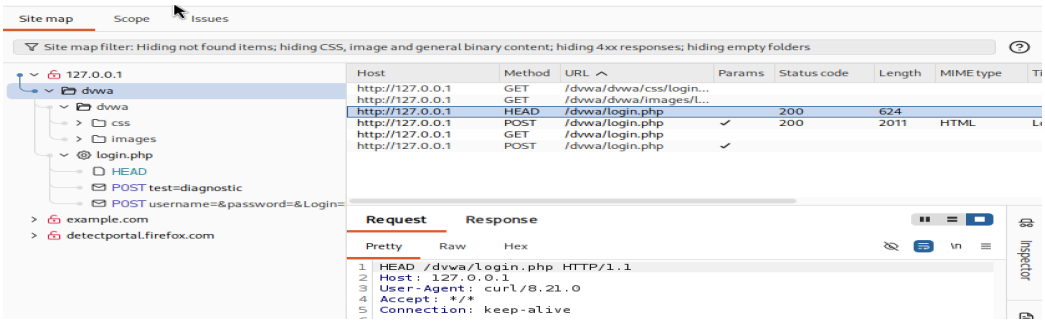


Figure 4: Burp Suite Site Map showing the DVWA login endpoint, HTTP request methods, and associated web resources.

HTTP Request and Login Parameter Analysis

Burp Suite was used to examine the DVWA login page and identify the HTTP requests generated during login. The Site Map displayed GET and POST requests associated with login.php. The GET request was used to access the login page, while the POST request was used to submit login form data. The captured requests also showed parameters and related web resources, helping to understand how the application communicates with the server.

Identification of Web Application Endpoints

During reconnaissance, the DVWA application was explored to identify its available modules and endpoints. The application contains several modules designed to demonstrate common web vulnerabilities, including Brute Force, Command Injection, SQL Injection, File Upload, and Cross-Site Scripting (XSS). These modules provide different input fields and functionalities that can be examined during security testing.

Task 24: Vulnerability Testing Using Burp Suite

Burp Suite Community Edition was used to prepare for security testing of the DVWA web application. The Repeater tool allows HTTP requests to be modified and resent to examine how the application responds. Testing was conducted in the local DVWA lab to investigate potential SQL injection, Cross-Site Scripting (XSS), and parameter manipulation vulnerabilities

1. SQL Injection Testing

Objective: To check whether the application handles user input securely when interacting with the database.

Testing Procedure: Burp Suite Repeater was used to modify the id parameter in a request to the DVWA SQL Injection module. A test input was submitted, and the server response was examined.

Observation: The application returned multiple user records for the modified input, indicating that the SQL Injection vulnerability could be exploited at the tested security level.

Recommendation: Use parameterized queries, validate user input on the server side, and apply least-privilege database permissions.

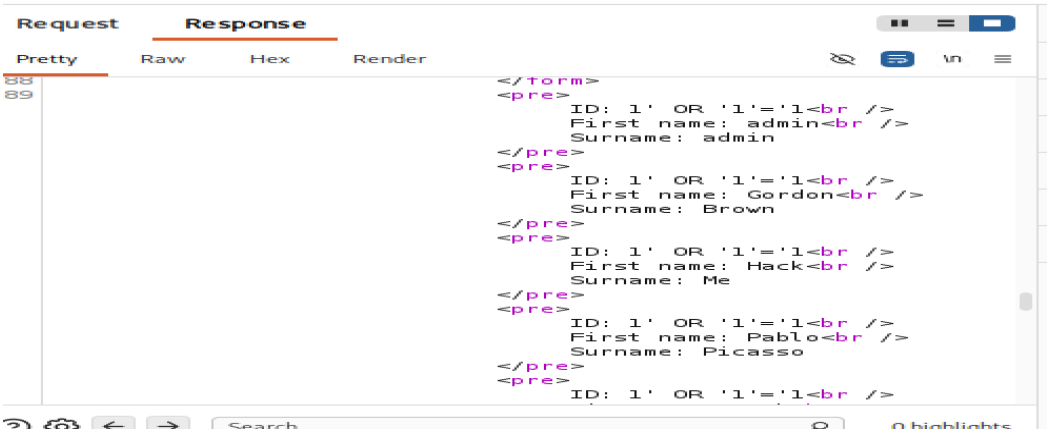


Figure 5: SQL Injection Testing Using Burp Suite Repeater

2. Cross-Site Scripting (XSS) Testing

Objective: To check whether the web application safely handles user input and prevents potentially harmful scripts from executing in a user's browser.

Testing Procedure: The XSS module in the local DVWA environment was opened. Test input was entered into the available input field, and the application's response was examined to determine how the input was handled.

Observation: The application displayed the submitted input as Hello HelloDVWA123 on the page. This confirms that the input was reflected in the response. Further testing would be required to determine whether executable scripts are also processed.

Recommendation: Validate and sanitise user input, encode output according to its context, and implement an appropriate Content Security Policy (CSP).

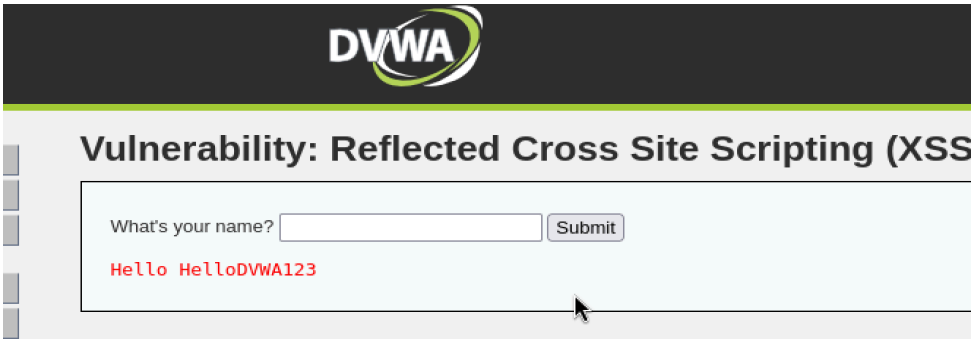


Figure 6: Reflected XSS Testing in DVWA

3. Authentication Testing Using Burp Suite

Objective: To examine how the web application handles login requests and manages authentication-related information.

Testing Procedure: Burp Suite was used to inspect the HTTP login request sent to the local DVWA application. The request method, submitted parameters, response status, and session-related cookies were examined.

Observation: The login request and server response were inspected to understand the authentication process. Any weaknesses should be reported only if they were observed during testing.

Recommendation: Use strong password policies, secure session management, HTTPS, and appropriate protections against repeated login attempts.

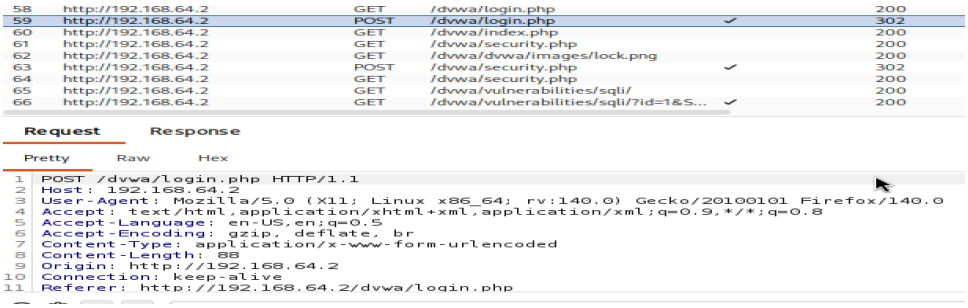


Figure 7: Authentication Request Analysis Using Burp Suite

4. Parameter Tampering Testing Using Burp Suite

Objective: To examine how changing a request parameter affects the response returned by the web application.

Testing Procedure: Burp Suite Repeater was used to modify the id parameter in a request to the local DVWA SQL Injection module. The original request was sent with id=1, then the parameter was changed to id=2. The responses were compared to observe differences in the returned user records.

Observation: Changing the id parameter was used to examine whether the application returned to different user records. The results helped demonstrate how user-supplied parameters influence the application's response.

Recommendation: Validate user-supplied parameters on the server side, use parameterised SQL queries, and enforce authorisation checks to prevent unauthorized access to demand.

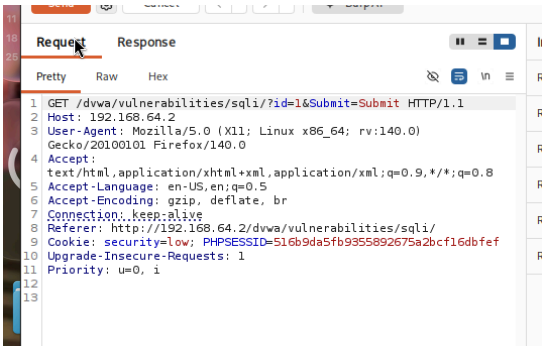


Figure 7: Authentication Request Analysis Using Burp Suite

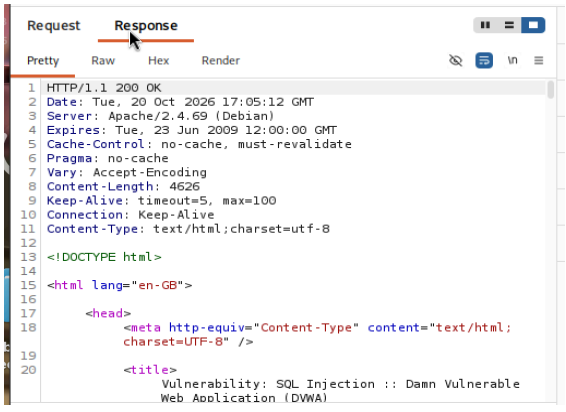


Figure 8: Parameter Tampering Using Burp Suite Repeater

5. Findings and Mitigation

Finding 1: SQL Injection

Description**:** Testing in the local DVWA environment showed that the SQL Injection module returned multiple user records when a modified input was submitted.

Risk Level: High at the tested security level.

Mitigation**:** Use parameterised SQL queries, validate input on the server side, and apply least-privilege database permissions.

Finding 2: Cross-Site Scripting (XSS)

Description**:** A test string was submitted through the Reflected XSS module. The application's response was examined to determine how the input was displayed.

Risk Level: To be determined from the test result.

Mitigation**:** Apply context-appropriate output encoding, validate input, and implement a suitable Content Security Policy (CSP).

Finding 3: Parameter Tampering

Description: The id parameter was examined to determine how changing its value affected the application's response.

Risk Level: To be determined from the test result.

Mitigation**:** Validate parameters on the server side and enforce authorisation checks before returning user records.

Conclusion**:** This task provided practical experience with Burp Suite Community Edition for testing a local Damn Vulnerable Web Application (DVWA). SQL Injection testing demonstrated how modified input could affect database query results. Reflected XSS testing helped examine how user input was displayed, while parameter testing demonstrated how changing request values affected application responses. The exercise highlighted the importance of secure coding, input validation, output encoding, parameterised queries, and proper authorisation controls.

Task 25: Directory and Endpoint Discovery

Objective: To identify accessible directories and endpoints in the local DVWA web application and understand how discovering them helps security testers assess the application's attack surface.

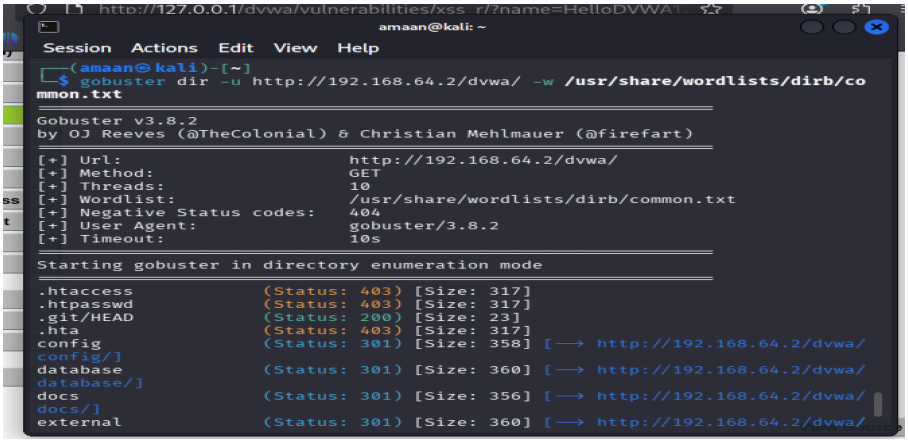


Figure 9: Directory and Endpoint Discovery Using Gobuster

Observation: Gobuster was used to discover directories and files within the local DVWA test environment. Several paths were identified with different HTTP status codes. A 200 status indicates a successful response, 301 indicates a redirect, and 403 indicates that access is forbidden. These results help identify paths that require further security review.

Recommendation: Disable directory listing where unnecessary, restrict access to sensitive directories, and ensure that configuration files and other sensitive resources cannot be accessed by unauthorized users.

Important: Gobuster has only identified these paths; this screenshot alone does not prove that a vulnerability exists.

Task 26: Vulnerability Assessment Report

Objective: To document the security testing performed on the local Damn Vulnerable Web Application (DVWA), identify potential web application vulnerabilities, assess their risk levels, and recommend suitable mitigation measures.

Tools Used:

1. Kali Linux
2. Burp Suite Community Edition
3. DVWA (Damn Vulnerable Web Application)
4. Gobuster
5. Firefox Browser

Testing Environment: A local, intentionally vulnerable DVWA application hosted at http://192.168.64.2/dvwa/.

Document the findings:

Vulnerability Findings and Risk Assessment:

Finding 1: SQL Injection

We tested the SQL Injection module in DVWA using Burp Suite.

Vulnerability Name: SQL Injection
Risk Level: To be determined from the observed results.

Description:
SQL injection occurs when an application incorporates untrusted user input into a database query without appropriate safeguards. An attacker may manipulate the query to access or modify data beyond the intended functionality.

Testing Procedure:
The DVWA SQL Injection module was accessed, and the id parameter was inspected using Burp Suite. Requests and responses were examined to understand how the application processed user-supplied input.

Potential Impact:
If the vulnerability is exploitable, it may allow unauthorized database access or disclosure of sensitive information.

Recommendation:
Use parameterised SQL queries, validate user input, and apply least-privilege permissions to database accounts.

Important: If you only changed id=1 to id=2, that demonstrates parameter manipulation, not SQL injection by itself. We should confirm the actual SQL injection result before marking it as a confirmed vulnerability.

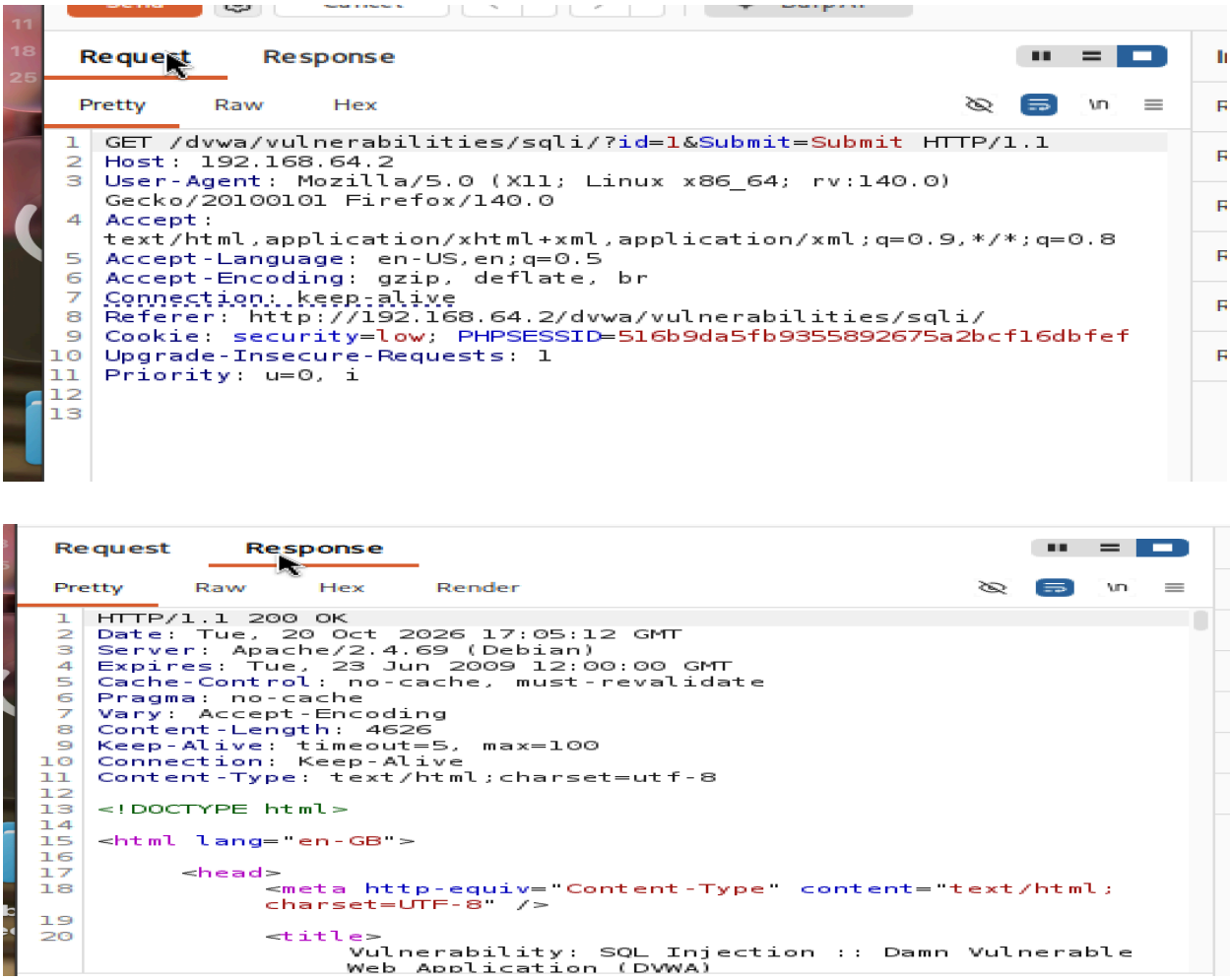


Figure 10: SQL Injection Request and Response Analysis Using Burp Suite Repeater

Finding 2: Reflected Cross-Site Scripting (XSS)

Vulnerability Name: Reflected Cross-Site Scripting (XSS)

Risk Level: Not confirmed — further testing required.

Description: Reflected Cross-Site Scripting (XSS) is a web security vulnerability in which an application returns untrusted user input in a webpage without properly encoding it. If the input is interpreted as executable JavaScript, it may affect users who open a specially crafted link or submit malicious input.

Testing Procedure: The Reflected XSS module in the local DVWA application was selected. User input was submitted, and the application response was to be examined to determine whether the input was safely handled or interpreted as executable JavaScript.

Observation:A JavaScript popup was not observed during the test. Therefore, the vulnerability has not been confirmed based on the available evidence.

Potential Impact: If exploitable, reflected XSS may allow an attacker to execute scripts in another user's browser, potentially affecting their session or exposing information available to the webpage.

Recommendation: Encode untrusted output according to its context, validate input where appropriate, and implement a suitable Content Security Policy (CSP).

Finding 3: Authentication Testing

Vulnerability Name: Authentication Security Testing

Risk Level: Not confirmed — further testing required.

Description: Authentication testing evaluates whether a web application verifies user identities correctly and protects login functionality against unauthorized access.

Testing Procedure: The login functionality of the local DVWA application was examined using Burp Suite. HTTP requests were inspected to understand how login credentials and session information were transmitted. Testing was performed within the local lab environment.

Observation: The login request was identified for analysis. However, no authentication bypass or unauthorized access was confirmed based on the available evidence.

Potential Impact: If authentication controls are weak, an attacker may gain unauthorized access to user accounts or protected application resources.

Recommendation: Use strong password policies, implement appropriate login-attempt limits, regenerate session identifiers after successful authentication, and enforce secure session management.

Conclusion: This assessment helped me understand web application security testing using Burp Suite and Gobuster in a local DVWA lab. I practiced analyzing HTTP requests and responses, testing input handling, examining authentication, modifying parameters, and discovering directories.